



Low-Power and Small-Area Inexact Multiplier for High-Speed Error-Resilient Applications (LSHIM)

Jainil Devaliya | 23110090@iitgn.ac.in
Adarsh Patidar | 23110012@iitgn.ac.in

Department of Electrical Engineering, Indian Institute of Technology Gandhinagar

1. Motivation

- In many real-world applications like image processing, machine learning, and DSP, perfect accuracy is not essential.
- Approximate computing allows faster, smaller, and energy-efficient hardware by tolerating small errors.
- Crucial for battery-powered devices such as IoT nodes, smartphones, and wearables.

Why Replace Traditional Multipliers?

- High Power Usage in exact multipliers
- Large Hardware Area due to complexity
- Slower Execution Time
- All of these are bottlenecks for modern embedded systems.

2. Existing Multiplier

Mitchell's Approximate Multiplier

- Based on logarithmic approximation
- Fast and compact, but has high error (MRED)

Others: ALM-SOA, REALM, HEALM, IALM

- Improve accuracy but need more power and area

3. Proposed Multiplier

What is LSHIM?

A novel approximate multiplier, optimizing:

- Power
- Speed
- Area
- While keeping error minimal

Key Features:

- Uses priority encoder and a custom result production unit
- Truncates final term for faster computation
- Combines shifts, MUXes, and logic efficiently

4. Algorithm

$$N = 2^k(1+x) \rightarrow N - 2^k = 2^k x$$

$$A = 2^{k_a}(1+x_a) \quad B = 2^{k_b}(1+x_b)$$

$$P = A \times B$$

$$P = 2^{k_a}(1+x_a) \times 2^{k_b}(1+x_b)$$

$$P = 2^{k_a+k_b}(1+x_a+x_b+x_a x_b)$$

$$P = 2^{k_a+k_b} + x_a 2^{k_a} 2^{k_b} + x_b 2^{k_b} 2^{k_a}$$

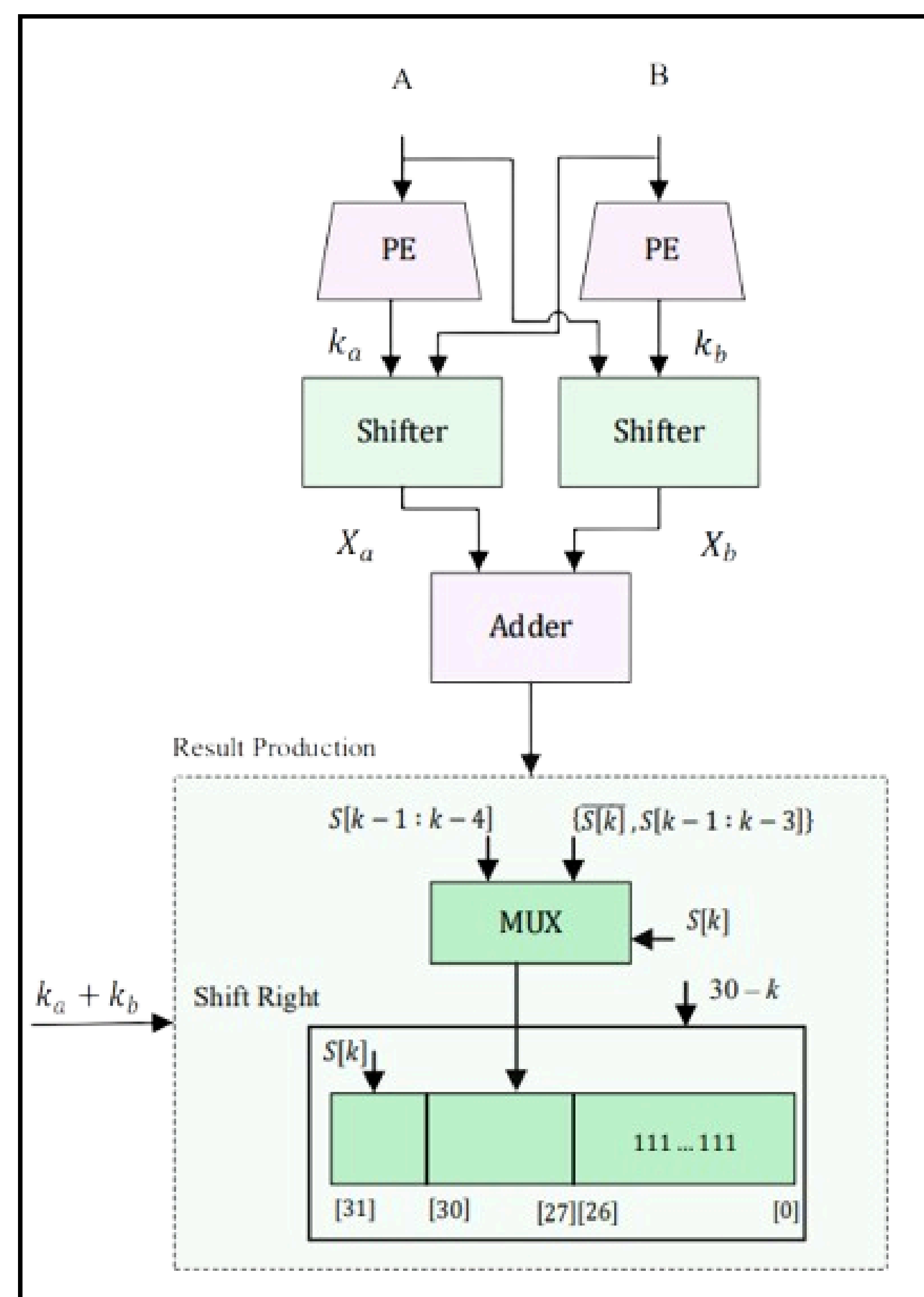
$$P = 2^{k_a+k_b} + 2^{k_b}(A - 2^{k_a}) + 2^{k_a}(B - 2^{k_b})$$

$$P = 2^{k_a+k_b} + A \times 2^{k_b} + B \times 2^{k_a} - 2 \times 2^{k_a+k_b}$$

$$P = A \times 2^{k_b} + B \times 2^{k_a} - 2^{k_a+k_b}$$

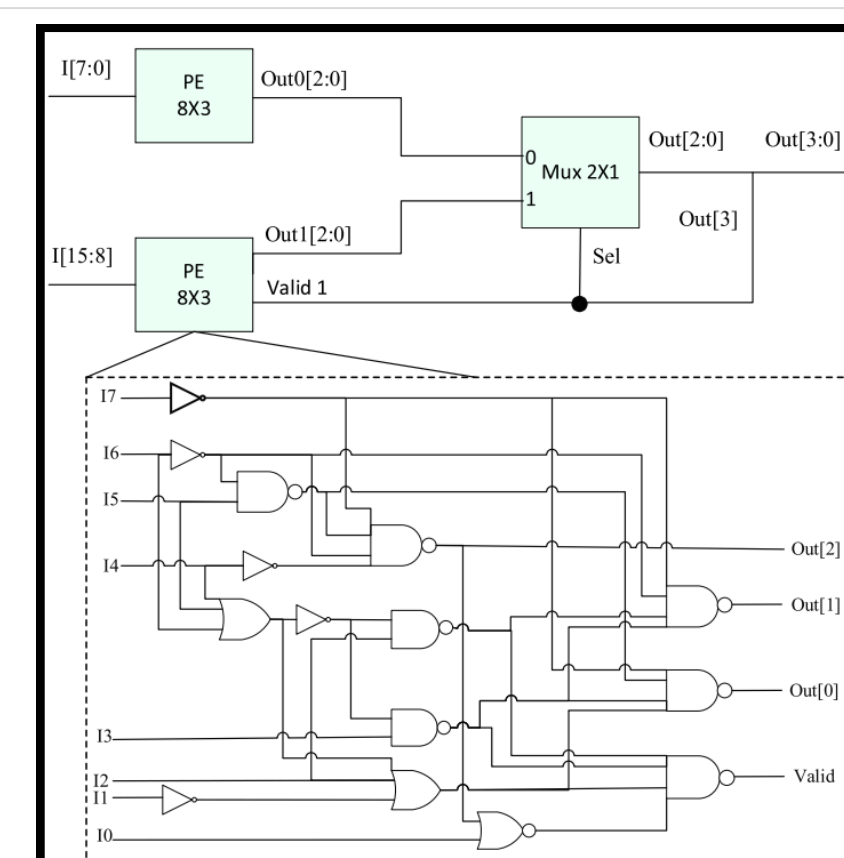
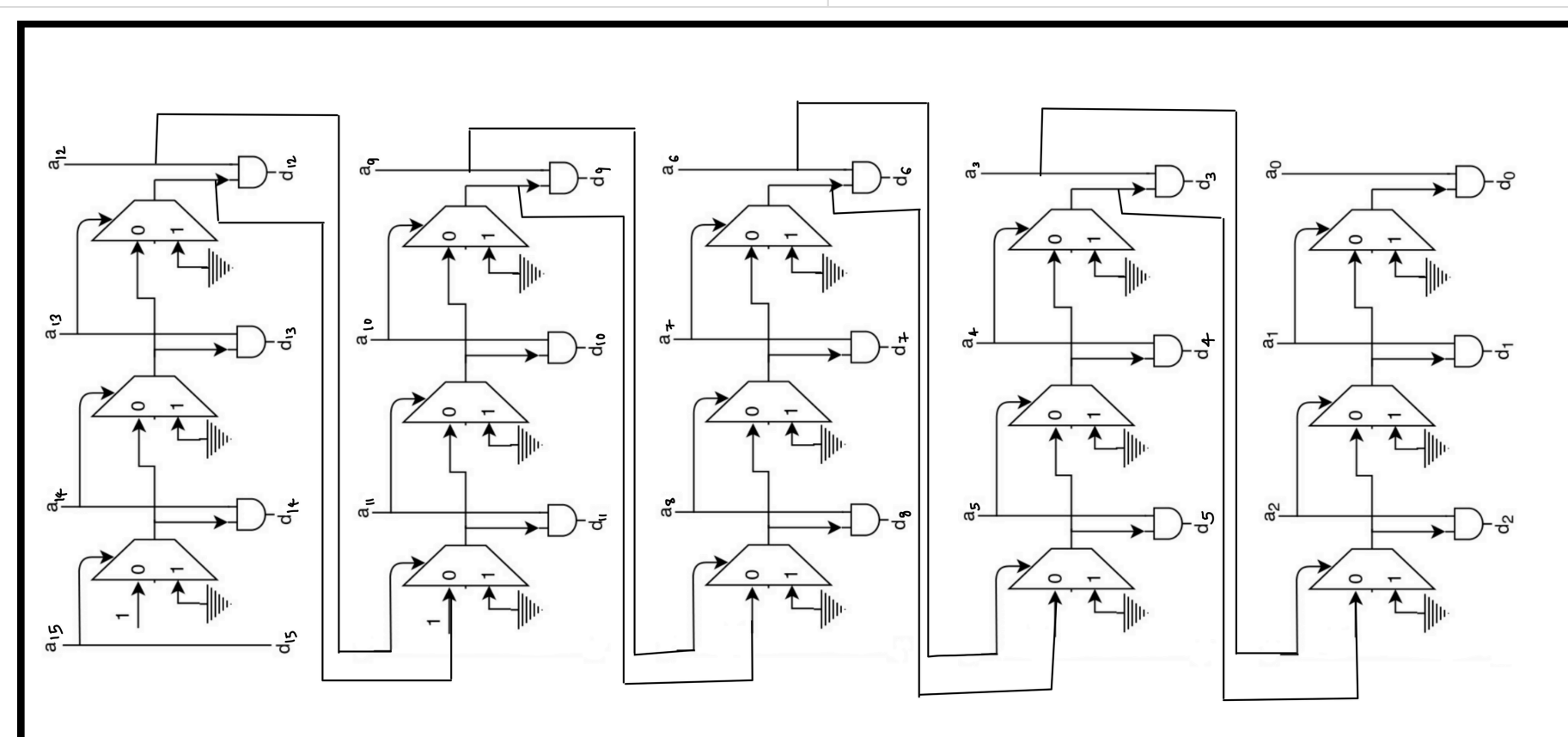
$$P = A \times 2^{k_b} + B \times 2^{k_a}$$

1. A, B : n -bits inputs, R : $2n$ -bits approximate result
2. Calculate k_a : priority encode A
3. Calculate k_b : priority encode B
4. Calculate X_a : shift A to the left by k_b bits
5. Calculate X_b : shift B to the left by k_a bits
6. Calculate $k=k_a+k_b$
7. Calculate $S=X_a+X_b$
8. if $S[k]=0$ then
9. $O_{MUX} = \{S[k], S[k-1:k-3]\}$
10. else
11. $O_{MUX} = \{S[k-1:k-4]\}$
12. end if
13. Calculate R : shift $\{S[k], O_{MUX}, 27'b1\}$ to the right by $(30-k)$



5. Leading One Detector V/S Priority Encoder

Metric	Leading One Detector	Priority Encoder
Dynamic Power	1.204 W	0.313 W
Static Power	0.073 W	0.071 W
Data Path Delay	6.626 ns	6.275 ns
Slice LUT	15	10
Slice Register	0	3



6. Simulation

Name	Value	0.000 ns	100.000 ns	200.000 ns	300.000 ns	400.000 ns
> A[1..	65535	65535	0	4433	31	43680
> B[15	65535	65535	0	16991	21207	43061
> O_...	42949672	4294967295	1	75497471	655359	1879048191

7. Examples

When S[k] = 0	When S[k] = 1
$A = 1010\ 1010\ 1010\ 0000$ $B = 1010\ 1000\ 0011\ 0101$ $K_a = 15$ $K_b = 15$ $K = 15 + 15 = 30$ $X_a = 1010\ 1010\ 1010\ 0000\ 0000\ 0000\ 0000\ 0000$ $X_b = 1010\ 1000\ 0011\ 0101\ 0000\ 0000\ 0000\ 0000$ $S = X_a + X_b$ $S = 1010\ 1001\ 0110\ 1010\ 1000\ 0000\ 0000\ 0000$ $S[K] = 0$ $O_{mux} = \{S[K], \sim S[K], S[K-1:K-3]\} = 01101$ $Result = \{O_{mux}, 27'b1111111111111111111111111111\} \gg (30-K)$ $Result = 01101111111111111111111111111111$	$A = 0001\ 1111$ $B = 0101\ 0010\ 1101\ 0111$ $K_a = 4$ $K_b = 14$ $K = 4 + 14 = 18$ $X_a = 0111\ 1100\ 0000\ 0000\ 0000$ $X_b = 0101\ 0010\ 1101\ 0111\ 0000$ $S = X_a + X_b$ $S = 1100\ 1110\ 1101\ 0111\ 0000$ $S[K] = 1$ $O_{mux} = \{S[K], S[K-1:K-4]\} = 10011$ $Result = \{O_{mux}, 27'b1111111111111111111111111111\} \gg (30-K)$ $Result = 1001\ 1111\ 1111\ 1111\ 1111$

8. Performance Comparison

Metric	Multiplier	Mitchell Multiplier	Proposed LSHIM Multiplier
Dynamic Power	39.984W	21.099 W	6.200 W
Static Power	0.485W	0.485 W	0.097 W
Data Path Delay	27.634ns	13.325 ns	24.161 ns
Slice LUT	361	173	274
Slice Register	0	0	6

9. Worst Case / Best Case / MRED / NMRD

Metric	Mitchell Multiplier	Proposed LSHIM Multiplier
Worst Case Error	11.11%	11.10%
Best Case Error	0%	0%
MRED	3.85%	2.90%
NMED	3.70%	2.73%

10. Applications

- Each pixel was convolved with a 3×3 Gaussian kernel, replacing standard multiplication with our custom approximate multiplier.
- Despite reduced computational precision, the visual quality was preserved, and the system achieved significant savings in power and area.
- Results confirm that the Proposed Multiplier is well-suited for image processing applications, where slight inaccuracies are perceptually negligible.

Original Image



Gaussian Blurred Image using Accurate Multiplier



Gaussian Blurred Image using Proposed Multiplier

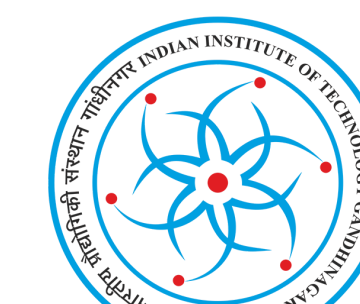
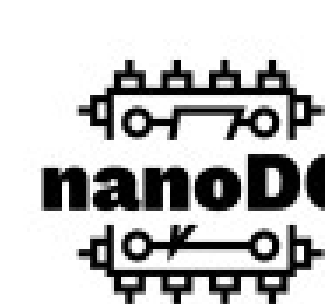


- SSIM: 0.9436697787237218
- PSNR: 33.39788694879294

11. Conclusion

LSHIM achieves an excellent balance between performance and accuracy, making it a compelling choice for error-resilient applications. It offers a remarkable 31.7% reduction in area, an impressive 84.43% decrease in power consumption, and is 36.1% faster than traditional multipliers. With a low Mean Relative Error Distance (MRED) of just 2.9%, LSHIM delivers efficient and reliable results, making it ideal for real-world systems where minor errors are acceptable in exchange for significant gains in speed, size, and energy efficiency.

12. Acknowledgement



We wish to express our sincere gratitude for the opportunity, guidance, and support extended by Professor Joyce Meki throughout the project course.